

UNIX Shell Programming — Last-Minute Cram Sheet

24CSAEC39 / CSAEC310 · Sem III · 50 marks · 2 hrs | **Part A** 10×1, no choice, 2 per unit | **Part B** 5×8, one full question per unit from a choice of two

Before anything else: your past papers are from an **older syllabus**. Their Unit V was grep/regex/sed — in **your** paper grep sits in Unit IV and **Unit V is shell scripting**, with zero past-paper history. **Skip sed and awk entirely — not in your syllabus**. If you only have an hour, do Unit V last but do it: it's 8 marks you'd otherwise drop cold.

UNIT I — Introduction to UNIX

PART B · Q1 or Q2 · 8 marks | PART A · Q1, Q2

Q1 = diagram + theory. Q2 = commands + permissions. Prepare all four below and either choice is safe.

LOCK UNIX architecture / kernel-shell relationship — explain with a neat diagram

Part B · Q1(a) · 4–5 marks · appeared Apr 23 + Mar 24

Draw concentric circles: Hardware (centre) → Kernel → Shell → Applications/Users (outer).

Kernel: core of the OS, loaded at boot, stays in memory. Manages memory, processes, files, devices. Only the kernel talks to hardware. Programs reach it through **system calls**.

Shell: the command interpreter between user and kernel. Reads command → interprets → passes to kernel → returns output. Shell is a user program, so many can exist: **Bourne (sh), C (csh), Korn (ksh), Bash**.

Mention the **shell interpretive cycle**: prompt → read → parse → locate command in `PATH` → fork/exec → wait → prompt again.

LOCK File permissions + chmod using absolute AND relative methods

Part B · Q2 · 3–4 marks · Part B 2 of 3 papers, Part A 3 of 3

9 bits = 3 sets (**owner / group / others**) × **rwX**. `ls -l` shows `-rwxr-xr-x`.

Absolute (octal): `r=4, w=2, x=1` → add them. `chmod 755 file` → `rwxr-xr-x`. `chmod 644` → `rw-r--r--`. `chmod 744` → `rwxr--r--`.

Relative (symbolic): 3 components — *who* (`u/g/o/a`) + *operator* (`+ - =`) + *permission* (`r/w/x`). `chmod u+x file`, `chmod go-w file`, `chmod a=r file`.

Directory meaning: `r` = list it, `w` = create/delete files in it, `x` = cd into it. Default file perms **666 (rw-rw-rw-)**, directories **777**, masked by `umask`.

LOCK Explain the following commands with syntax, options and example (*list changes every year*)

Part B · Q1(b) or Q2(a) · 4–5 marks · 3 of 3 papers

Format each as **syntax + 2 options + 1 example** — that's how marks are awarded. Most-asked set:

`echo` display text (`-n` no newline, `-e` escapes) · `ls` (`-l` long, `-a` hidden, `-R` recursive, `-t` by time) · `who` logged-in users (`-H` heading, `am i`) · `date` (`+%d-%m-%Y` format) · `passwd` change password · `cp` (`-i` interactive, `-r` recursive) · `mv` rename/move (`-i`) · `rm` (`-i`, `-r`, `-f`) · `cat` (`-n` number, `-s` squeeze) · `wc` (`-l` lines, `-w` words, `-c` chars) · `mkdir` (`-p`) · `rmdir` · `pwd` · `uname` (`-a`, `-r`) · `man` · `tar` (`-cvf`, `-xvf`, `-tvf`) · `gzip` / `gunzip`

BACKUP vi editor modes · Salient features of UNIX

Part B · Q1 · 4 marks each · vi = Aug 23, features = Mar 24

vi — 3 modes: **Command mode** (default; navigation `h j k l`, `dd` delete line, `yy` copy, `p` paste, `x` delete char) · **Insert mode** (enter with `i a o I A O`, leave with **Esc**) · **ex / Escape mode** (`:w` save, `:q` quit, `:wq` save+quit, `:q!` force quit, `:set nu`).

Features: multi-user, multitasking, portability (written in C), hierarchical file system, security via permissions, the shell as programmable interface, pipes & filters (building-block approach), communication utilities, large tool set.

PART A · Q1, Q2 — MEMORISE

Shell is both a **scripting language and an interpreter** · Default ordinary-file permission **rw-rw-rw-** · `755` = `rwxr-xr-x`, `744` = `rwxr--r--` · Relative-mode expression has **3 components**

★ ANSWER Q3. The session diagram is 3-for-3 in the Q3(a) slot and always pairs with a process question in 3(b).

HIGHEST CONFIDENCE ON THE PAPER Illustrate the various stages of a UNIX session with a neat diagram

Part B · Q3(a) · 4–5 marks · ALL 3 PAPERS, same slot, near-identical wording

Draw as a flow: Login prompt → enter username → password prompt (not echoed) → verify against `/etc/passwd` and `/etc/shadow` → if invalid, back to login → valid: shell starts, reads `/etc/profile` then `.profile` / `.bash_profile` → \$ prompt → work phase: shell interpretive cycle repeats (read → parse → execute → display) → logout via `exit` or **Ctrl+D** → back to login prompt.

Three stages to name: **Login** → **Work/Session** → **Logout**.

LOCK Process utilities — types of process, initialization & termination, `nice` / `renice`

Part B · Q3(b) · 4–5 marks · 3 of 3 papers, angle rotates — cover all three

Types: interactive/foreground, background (`&`), daemon (system background, e.g. `init`, `cron`), zombie (finished but parent hasn't collected status), orphan (parent died, adopted by `init`).

Init & termination: `fork()` creates a child (copy of parent), `exec()` overlays it with the new program, parent `wait()`s, child `exit()`s returning status. **init** is **PID 1**, ancestor of every process. Boot order: `init` → `getty` → `login` → shell.

Priority: **nice value -20 (highest priority) to +19 (lowest)**, default 0. `nice -n 10 command` starts a command at lower priority. `renice -n 5 -p PID` changes a **running** process. Only **root** can raise priority (negative nice).

Related: `ps` (-f full, -e all, -u user), `kill PID`, `kill -9`, `jobs`.

BACKUP · Q4 `du` with options · text processing commands · networking commands

Part B · Q4 · 3–5 marks · `du` 2 of 3, text processing 2 of 3, networking Mar 24

du disk usage per directory: `-a` all files not just dirs, `-s` summary total only, `-h` human readable, `-k` in KB. (`df` = free space on file systems, `-h`, `-i` inodes.)

Text processing: `head` `tail` `cut` `paste` `sort` `uniq` `tr` `wc` `grep` — name it, give syntax, one example.

Networking: `ping` tests reachability (ICMP echo) · `host` DNS name → IP lookup · `tracpath` / `traceroute` shows the route hop by hop ·

`netstat` network connections and statistics · `route` **manipulates the TCP/IP routing table** · also `ftp`, `telnet`, `ssh`, `finger`, `ifconfig`.

PART A · Q3, Q4 — MEMORISE

`ps` shows process attributes · `tar -c` create archive, `-x` extract, `-t` list, `-f` filename · `ping` = reachability · `route` = TCP/IP routing table · file comparison = `cmp` / `comm` / `diff`

★ ANSWER Q5. Streams/redirection is 3-for-3 in the Q5(a) slot. The (b) partner rotates between tee / variables / job states — all three are below.

HIGHEST CONFIDENCE Explain the three standard streams / standard redirection files of UNIX

Part B · Q5(a) · 3–5 marks · ALL 3 PAPERS, same slot

Every process opens three files by default:

stdin — **fd 0** — standard input — default **keyboard**

stdout — **fd 1** — standard output — default **terminal/screen**

stderr — **fd 2** — standard error — default **terminal/screen**

Redirection: `>` output, overwrite · `>>` output, append · `<` input · `2>` errors · `2>>` append errors · `> file 2>&1` both to one file · `/dev/null` discards.

Examples: `ls -l > out.txt` · `wc -l < file` · `cmd 2> err.log` · `sort < in.txt > out.txt`

Diagram: a box labelled *command*, arrow 0 going in, arrows 1 and 2 coming out.

LOCK The **tee** command — demonstrate it in a pipeline, with a diagram

Part B · Q5(b) or Q6(a) · 4–5 marks · 2 of 3 papers + a Part A MCQ

`tee` reads standard input and writes it to **standard output AND to one or more files at the same time** — it's a T-junction in a pipe. Lets you save intermediate output without breaking the pipeline.

Syntax: `tee [-a] file` (`-a` appends instead of overwriting)

`ls -l | tee list.txt | wc -l` — saves the listing *and* passes it on to `wc`.

Diagram: input arrow → **tee** box → two output arrows, one to *screen*, one to *file*.

HIGH Types of variables / environment variables · alias · command substitution

Part B · Q5(b) or Q6(b) · 3–4 marks · one of these three fills the slot

Variables: **local/shell** — `x=10`, `echo $x`, exists only in the current shell. **Environment** — exported, inherited by child processes. **No spaces around =**.

Standard ones: `PATH` (search path for commands) `HOME` `SHELL` `USER` / `LOGNAME` `PS1` (primary prompt) `PS2` (continuation prompt) `TERM` `MAIL`. `set` lists all · `env` lists environment · `export x` promotes local → environment · `unset x` removes · `readonly x`.

alias: a short name for a long command. `alias ll='ls -l'` · `unalias ll` · `alias` alone lists all. Put them in `.profile` / `.bashrc` to persist.

Command substitution: replaces a command with *its output*. Backquotes ``date`` or `$(date)`. `echo "Today is $(date)"` · `echo "You have $(ls | wc -l) files"`.

IF ASKED Job control — job states diagram, **bg** / **fg**, signals

Part B · appeared in all 3 papers, but your syllabus files this under Unit II process utilities — expect it one unit earlier

States: Running (foreground) ⇌ Running (background) ⇌ Stopped/Suspended → Terminated.

`cmd &` start in background · **Ctrl+Z** suspend the foreground job (sends **SIGTSTP**) · `bg %1` resume a stopped job in background · `fg %1` bring a background job to foreground · `jobs` list with job numbers · `kill %1`.

Signals: **SIGINT** (Ctrl+C, interrupt) · **SIGTSTP** (Ctrl+Z, suspend, catchable) · **SIGSTOP** (suspend, **cannot be caught or ignored**) · **SIGCONT** (resume) · **SIGKILL** (kill -9, cannot be caught).

PART A · Q5, Q6 — MEMORISE

`|` connects streams (pipe) · `&` pushes a process to background · `tee` copies stdin to stdout *and* a file · output can be redirected by `|`, `>` **and** `>>` (the "all of these" option is usually correct) · **Ctrl+Z** suspends

★ DENSEST UNIT — it absorbed the old Unit V. Q7 ≈ filter family. Q8 ≈ tr / uniq / grep + regex. `sed` and `awk` are OUT.

LOCK · FIXED 5-MARK FORMULA Describe the five categories of operators used in regular expressions

Part B · Q7 or Q8 · 5 marks · appeared twice, 5 marks both times, same phrasing

1 · Atoms — *what* to match: `.` any one character · `[abc]` character class · `[^abc]` negated class · `[a-z]` range · a literal character · `\` escapes a metacharacter.

2 · Quantifiers / repetition — *how many*: `*` zero or more of the preceding · `+` one or more · `?` zero or one · `{m,n}` m to n times.

3 · Anchors / position — *where*: `^` start of line · `$` end of line. `^$` matches an empty line.

4 · Alternation & grouping: `|` or `( )` group.

5 · Character classes: `[:alpha:]` `[:digit:]` `[:upper:]` `[:lower:]` `[:space:]` `[:alnum:]`.

Give one example per category — e.g. `grep "^a.*z$" file`.

LOCK What is `grep`? Explain the `grep` operation with a flow chart · `grep` vs `egrep` vs `fgrep`

Part B · Q7 or Q8 · 3–5 marks · Part B 3 of 3, Part A 3 of 3

grep = Global Regular Expression Print. Scans input **line by line**, prints every line matching the pattern. Syntax: `grep [options] pattern filename(s)`. Quote the pattern.

Flow chart: Start → open file → read next line → *does it match the pattern?* → YES: print the line → NO: skip → *more lines left?* → YES: loop back → NO: stop.

Options: `-i` ignore case · `-v` invert, show non-matching · `-c` count of matching lines only · `-n` prefix line numbers · `-l` filenames only · `-w` whole word · `-x` whole line · `-e` multiple patterns · `-E` extended regex.

grep = basic regex (BRE) · **egrep** = extended regex, `+` `?` `|` `( )` work without backslashes, = `grep -E` · **fgrep** = fixed strings only, no regex at all, fastest, = `grep -F`.

LOCK Explain the `tr` command with its options · `uniq` with its options

Part B · Q8 · 4–5 marks · tr 2 of 3 in the Q8(a) slot, uniq 1 of 3 + 2 Part A MCQs

tr — **translate characters**. Key exam point: **it takes input only from standard input** — no filename argument, so you must redirect or pipe.

`tr 'a-z' 'A-Z' < file` lowercase → uppercase · `tr -d '0-9' < file` delete digits · `tr -s ' ' < file` squeeze repeated characters into one · `tr -c` complement the set · `echo "hello" | tr 'a-z' 'A-Z'` → HELLO

uniq — **needs SORTED input** (it only compares *adjacent* lines). `-u` only non-repeated lines · `-d` only duplicated lines · `-c` prefix each line with its count · `-i` ignore case. Typical: `sort file | uniq -d`.

HIGH · Q7 The filter family — `cat`, `head`, `tail`, `cut`, `paste`, `sort`, `wc`

Part B · Q7 · 3–5 marks · appears in all 3 papers, subset changes each time

cat — concatenate	<code>-n</code> number lines, <code>-s</code> squeeze blank lines, <code>-v</code> show non-printing. <code>cat f1 f2 > f3</code> joins files. <code>cat > new</code> creates.
head — beginning	<code>head -n 5 file</code> or <code>head -5 file</code> → first 5 lines (default 10)
tail — end	<code>tail -n 5 file</code> last 5 lines · <code>tail +10</code> from line 10 on · <code>-f</code> follow a growing file
cut — columns	<code>-c</code> by character, <code>-f</code> by field, <code>-d</code> sets delimiter (default Tab). <code>cut -d: -f1,3 /etc/passwd</code> · <code>cut -c -3</code> = columns 1–3
paste — join	Joins files side by side , line for line. <code>-d</code> sets the delimiter. <code>paste f1 f2</code>
sort	<code>-n</code> numeric, <code>-r</code> reverse, <code>-k</code> field number, <code>-t</code> delimiter, <code>-u</code> unique. Default delimiter = one or more spaces.
wc — count	<code>-l</code> lines, <code>-w</code> words, <code>-c</code> characters. <code>wc -l file</code>
compare files	<code>cmp</code> byte by byte, exit status 0 if identical · <code>comm</code> needs sorted files, 3 columns · <code>diff</code> shows what to change (<code>-c</code> context, <code>-u</code> unified, <code>-i</code> ignore case)

PART A · Q7, Q8 — MEMORISE

`cut -c` characters, `-f` fields, `-d` delimiter · `cut -c -3` = columns 1–3 · `uniq -d` repeated, `-u` non-repeated, `-c` count · `sort` default delimiter = **one or more spaces** · `cmp` exit status 0 if files identical · `grep -i` ignore case, `-c` count, `-v` invert · `*` matches the **immediately preceding character**

⚠ NO PAST-PAPER HISTORY — predicted from the syllabus topic order. Q9 ≈ input / arguments / conditionals. Q10 ≈ case / expr / loops / shift. This is 8 marks you cannot afford to leave blank.

TOP PICK · Q9 test and [] to evaluate expressions + the if conditional

Part B · Q9(a) · 4–5 marks · first-named construct in the syllabus after read

Spaces inside [] are mandatory. [\$a -gt \$b] — never [\$a-gt\$b] .

Numeric: -eq -ne -gt -ge -lt -le · **String:** = equal, != not equal, -z zero length, -n non-zero length

File: -f regular file, -d directory, -e exists, -r -w -x readable/writable/executable, -s size > 0

Logical: -a AND, -o OR, ! NOT

```
if [ $a -gt $b ]
then
    echo "$a is greater"
elif [ $a -eq $b ]
then
    echo "equal"
else
    echo "$b is greater"
fi
```

TOP PICK · Q10 The case conditional + expr for computation and string handling

Part B · Q10(a) · 4–5 marks · the classic menu-driven-script question

expr — spaces are mandatory; escape *. expr 5 + 3 → 8 · expr 10 * 2 → 20 · expr 10 / 3 → 3 · expr 10 % 3 → 1

Strings: expr length "hello" → 5 · expr substr "hello" 2 3 → ell · expr index "hello" l → 3

Assign with command substitution: sum=\$(expr \$a + \$b)

```
echo "1.Add 2.Sub 3.Mul 4.Div"
read ch
read a b
case $ch in
    1) echo "Sum = `expr $a + $b`" ;;
    2) echo "Diff = `expr $a - $b`" ;;
    3) echo "Prod = `expr $a \* $b`" ;;
    4) echo "Div = `expr $a / $b`" ;;
    *) echo "Invalid choice" ;;
esac
```

HIGH Command line arguments, exit status, && and ||

Part B · Q9(b) · 3–4 marks · three syllabus lines, one answer

Positional parameters: \$0 script name · \$1 ... \$9 the arguments · \$# number of arguments · \$* all arguments as one string · @\$ all as separate words · \$? **exit status of the last command** · \$\$ PID of the current shell.

Exit status: every command returns one. 0 = **success**, non-zero = failure. Read it with \$? — *immediately*, it is overwritten by the next command.

exit n ends a script with status n.

Logical operators: cmd1 && cmd2 runs cmd2 **only if cmd1 succeeded**. cmd1 || cmd2 runs cmd2 **only if cmd1 failed**.

grep "unix" file && echo "Found" || echo "Not found"

read: read name takes one line from the keyboard. read a b c splits on whitespace. echo -n "Enter name: " then read name makes it interactive.

HIGH while / for looping · set and shift · \$0

Part B · Q10(b) · 3–4 marks · last three syllabus lines of the unit

set assigns positional parameters: set 10 20 30 → \$1=10, \$2=20, \$3=30. set `date` → \$1=day, \$2=month...

shift moves every parameter one place left — \$2 → \$1, \$3 → \$2 — and \$# drops by one. shift n shifts by n. Lets a script walk more than 9 arguments.

\$0 holds the name the script was *invoked* by. Link one script under several names (ln script other) and branch on case \$0 in ... so one file behaves differently per name.

```
i=1
while [ $i -le 5 ]
do
    echo $i
    i=`expr $i + 1`
done

for f in *.txt
do
    echo "File: $f"
done
```

WRITE-A-SCRIPT Three you should be able to reproduce cold

The lab component is shell scripts — expect at least one "write a script to..."

Largest of three numbers

```
echo "Enter 3 numbers"
read a b c
if [ $a -gt $b -a $a -gt $c ]
then
    echo "$a is largest"
elif [ $b -gt $c ]
then
    echo "$b is largest"
else
    echo "$c is largest"
fi
```

Even or odd / factorial

```
echo "Enter a number"
read n
r=`expr $n % 2`
if [ $r -eq 0 ]
then
    echo "$n is even"
else
    echo "$n is odd"
fi

f=1 ; i=1
while [ $i -le $n ]
do
    f=`expr $f \* $i`
    i=`expr $i + 1`
done
echo "Factorial = $f"
```

Count and print all arguments — uses \$# + shift

```
echo "Number of arguments: $#"
```

```
while [ $# -gt 0 ]
do
    echo "Argument: $1"
    shift
done
```

Marking traps: start with `#!/bin/sh` · use `expr` for arithmetic, **not** `$( ( )` · use `[ ]`, **not** `[[ ]]` · close with `fi`, `done`, `esac` · `;;` ends every `case` branch · no spaces around `=` when assigning, but spaces *required* inside `[ ]` and around `expr` operators.

PART A · Q9, Q10 — MEMORISE

`$?` = exit status of the last command, **0 = success** · `$#` = number of arguments · `$0` = script name · `shift` moves positional parameters left · `fi` closes `if`, `esac` closes `case`, `done` closes loops · `-eq` is numeric, `=` is string · `&&` runs on success, `||` on failure
If the paper still uses the old mapping these two slots will be grep/regex instead — that's Unit IV above, so you're covered either way.

If you have 45 minutes: the four diagrams (UNIX session · standard streams · architecture · grep flow chart), then `chmod` absolute vs relative, then the five regex categories, then the Unit V syntax blocks. That is roughly 30 of the 40 Part-B marks in reach.

Honest note: Unit I–IV predictions come from three real papers. Unit V has **no precedent** — it is derived from your syllabus text, so treat it as a strong guess, not a guarantee.